xGEMS tutorial 1 — basics for a hydrothermal geochemist

Chemical system: MORB-type basalt glass + seawater, initialised from `gems_files/SW-B_titr3-dat.lst`.

This notebook walks through the everyday `xgems` functionality using the **dictionary interface** (`ChemicalEngineDicts`) and the **`Material`** helper for building recipes. It mirrors the structure of the official `gemshub/xgems-jupyter` `basics_xgems.ipynb` and the `Material` demo, but is themed around water–rock interaction.

> **How to run this.** These cells are written against the real API but were *not* executed for you (they need your local `xgems` conda environment and your `SW-B_titr3-dat.lst` GEMS3K export). Create the environment from the template repo's `environment.yml` (`conda env create -f environment.yml`), place your exported `gems_files/SW-B_titr3-dat.lst` (+ the `-dch`, `-ipm`, `-dbr` files it references) as referenced below, then Run All.

Contents

0. Imports

`ChemicalEngineDicts` is the name-keyed interface: every getter returns a Python `dict` keyed by element / species / phase name (no index bookkeeping). `Material` turns human recipes (formulas, grams, oxides) into the bulk-composition vector **b** that GEM minimisation needs.

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from xgems import ChemicalEngineDicts, Material

# Path to your GEM-Selektor standalone export. Everything downstream keys off this.
LST = "gems_files/SW-B_titr3-dat.lst"
```

1. Initialise, re-equilibrate, return codes, print

`ChemicalEngineDicts(LST)` loads the chemical system (elements, species, phases, thermodynamic data and activity models) exactly as defined in GEM-Selektor. `reequilibrate()` recomputes the Gibbs-energy-minimum state.

GEMS return / calculation codes (returned as a short string here):

mode	meaning
warm start	SIA — smart initial approximation, reuses the previous speciation
cold start	AIA — automatic (LPP) initial approximation from scratch

`0` no recalculation needed · `1` need AIA · `2` OK after AIA · `3` suspect after AIA · `4` AIA failure · `5` need SIA · `6` OK after SIA · `7` suspect after SIA · `8` SIA failure · `9` terminal GEMS3K error (restart required). A trustworthy result is **2** (cold) or **6** (warm).

In [3]:
```
gems = ChemicalEngineDicts(LST)

status = gems.reequilibrate()          # warm start by default
print("re-equilibration status:", status)

# NOTE on API style: in ChemicalEngineDicts the *output getters* (pH, phase_names,
# phases_volume, phase_sat_indices, ...) are exposed as ATTRIBUTES (no parentheses).
# Actions/setters (equilibrate, set_bulk_composition, suppress_phase, ...) are called
# with (). If a getter raises "method not callable" style errors,
# simply add () to it.
print("pH             :", gems.pH)
print("Eh (V)         :", gems.Eh)
print("ionic strength :", gems.ionic_strength, "mol/kg")
print("system mass    :", gems.system_mass, "kg")
print("system volume  :", gems.system_volume, "m3")
```

```
re-equilibration status: OK after GEM calculation with LPP AIA
pH             : 5.736364839778835
Eh (V)         : -0.24873784250742578
ionic strength : 0.48436210410104147 mol/kg
system mass    : 1.0500000000000005 kg
system volume  : 0.0011198598456421051 m3
```

A full human-readable dump of the state is handy for eyeballing or for pasting into an LLM to ask questions about the result:

In [4]:
```
with open("result_initial.txt", "w") as f:
    f.write(repr(gems))          # same content as GEM-Selektor's multi-page print
print("wrote result_initial.txt")
```

```
wrote result_initial.txt
```

2. Extract data from the equilibrium state

Because getters are name-keyed dicts, pulling results into a `pandas` table is trivial. Below: a phase-level summary (amount, mass, volume, saturation index) and the aqueous molalities of the elements.

In [5]:
```
# --- phase-level summary ------------------------------------------------
phase_df = pd.DataFrame({
    "moles"      : gems.phases_moles,
    "mass_kg"    : gems.phases_mass,
    "volume_m3"  : gems.phases_volume,
    "vol_frac"   : gems.phases_volume_frac,
    "SI"         : gems.phase_sat_indices,     # log10(IAP/K): 0 = at equilibrium
}).sort_values("mass_kg", ascending=False)

# "present" phases = those with a non-trivial amount
present = phase_df[phase_df["moles"] > 1e-12]
present
```

Out[5]:

	moles	mass_kg	volume_m3	vol_frac	SI
aq_gen	5.436601e+01	9.953180e-01	1.100924e-03	9.830906e-01	-1.515020e-10
Chlorite	3.756845e-02	2.215843e-02	7.959969e-06	7.108004e-03	9.726927e-07
Cpx	4.337873e-02	1.034990e-02	2.932292e-06	2.618446e-03	-4.949874e-08
Plagioclase	3.133488e-02	8.227196e-03	3.155728e-06	2.817967e-03	3.944477e-08
Alkali Feldspar	3.133488e-02	8.227196e-03	3.155728e-06	2.817967e-03	3.944477e-08
Anhydrite	2.326920e-02	3.167929e-03	1.070616e-06	9.560264e-04	1.555472e-08
Titanite	8.116808e-03	1.591223e-03	4.534161e-07	4.048864e-04	5.924828e-13
Epidote	1.051693e-03	5.087928e-04	1.470592e-07	1.313193e-04	3.288575e-08
Galena	1.685005e-03	4.031661e-04	5.306081e-08	4.738165e-05	-3.187353e-08
Chalcocite	3.024765e-04	4.814186e-05	8.312053e-09	7.422405e-06	9.024078e-08
Gold	1.973542e-07	3.887227e-08	2.015974e-12	1.800202e-09	1.295638e-08

In [6]:
```
# --- aqueous chemistry ----------------------------------------------------
aq = pd.DataFrame({
    "molality_mol_per_kgH2O": gems.aq_elements_molality,
    "molarity_mol_per_L"    : gems.aq_elements_molarity
})
aq["log10_molality"] = np.log10(aq["molality_mol_per_kgH2O"].where(lambda s: s > 0))
aq.sort_values("molality_mol_per_kgH2O", ascending=False)
```

Out[6]:

	molality_mol_per_kgH2O	molarity_mol_per_L	log10_molality
H	1.110173e+02	9.702800e+01	2.045391
O	5.554341e+01	4.854439e+01	1.744633
Cl	5.682311e-01	4.966284e-01	-0.245475
Na	4.543568e-01	3.971034e-01	-0.342603
Ca	4.649827e-02	4.063903e-02	-1.332563
K	1.394067e-02	1.218401e-02	-1.855716
Si	9.013460e-03	7.877675e-03	-2.045108
Zn	6.451293e-03	5.638366e-03	-2.190353
S	3.096954e-03	2.706707e-03	-2.509065
C	2.043618e-03	1.786102e-03	-2.689600
Pb	2.844372e-04	2.485953e-04	-3.546014
Mg	8.236342e-05	7.198481e-05	-4.084266
Cu	3.502738e-05	3.061358e-05	-4.455592
Al	2.334644e-06	2.040456e-06	-5.631779
Ag	3.910193e-07	3.417470e-07	-6.407802
Fe	2.441037e-07	2.133442e-07	-6.612426
Au	9.032116e-09	7.893980e-09	-8.044210
Ti	1.822834e-09	1.593139e-09	-8.739253
Zz	-3.493782e-17	-3.053531e-17	NaN

In [7]:
```
# The full system recipe (b vector) as a dict — the master input to GEM:
b0 = gems.bulk_composition          # {element: moles}
print(b0)
```

```
{'Ag': 3.76237350018477e-07, 'Al': 0.142044162926922, 'Au': 2.06044908697781e-07, 'C':
0.00196636141681008, 'Ca': 0.119950310775163, 'Cl': 0.54674992467407, 'Cu': 0.00063865
6180318319, 'Fe': 0.073457124256494, 'H': 107.122020915768, 'K': 0.013455031811313, 'M
g': 0.158339405482984, 'Na': 0.501467141249838, 'O': 55.028866847164, 'Pb': 0.00195868
953418406, 'S': 0.0282365542413894, 'Si': 0.405840465505492, 'Ti': 0.00811680931010983
, 'Zn': 0.00620741025626573, 'Zz': 0.0}
```

3. Set a new composition / material / T / P and equilibrate

Rather than reusing the exported recipe, we build a fresh system from two `Material`s — a **leucogranite** (four rock-forming minerals, by mass) and a **saline water** (1 kg H_2O + 30 g NaCl) — add them together, and equilibrate. Two equivalent routes are shown:

- **stateful** — set `T`, `P`, then `set_bulk_composition(b)` and call `equilibrate()`;
- **one-shot** — `equilibrate(T, P, recipe)`.

`T` is in **kelvin**, `P` in **pascal**. We run at 100 °C / 50 bar and dump the resulting state to `result_granite.txt`.

In [8]:
```
granite = Material(gems, "leucogranite")
granite.add("KAlSi3O8",         17.0, "g")    # Microcline
granite.add("KAl3Si3O10(OH)2", 19.0, "g")    # Muscovite
granite.add("NaAlSi3O8",        29.0, "g")    # Albite
granite.add("SiO2",             35.0, "g")    # Quartz
```

Out[8]:
```
Material('leucogranite', 4 constituents, 100 g, 0.801888 mol)
```

In [9]:
```
water = Material(gems, "water")
water.add("H2O", 1000.0, "g")
water.add("NaCl", 30, "g")    # 30 g NaCl in ~1 kg water (a saline solution)
```

Out[9]:
```
Material('water', 2 constituents, 1030 g, 56.0217 mol)
```

In [10]:
```
T = 100.0 + 273.15      # K
P = 50e5                # Pa  (50 bar)

mix = granite+water

# --- stateful route ---
gems.T = T
gems.P = P
gems.set_bulk_composition(mix.b_dict())
status = gems.equilibrate()
print("stateful  ->", status, "| pH", round(gems.pH, 3))

# --- one-shot route (identical result) ---
status = gems.equilibrate(T, P, mix)
print("one-shot  ->", status, "| pH", round(gems.pH, 3))
```

```
stateful  -> OK after GEM calculation with LPP AIA | pH 6.684
one-shot  -> OK after GEM calculation with LPP AIA | pH 6.684
```

In [11]:
```
with open("result_granite.txt", "w") as f:
    f.write(repr(gems))          # same content as GEM-Selektor's multi-page print
print("wrote result_granite.txt")
```

```
wrote result_granite.txt
```

4. Build a `Material` — basalt glass

A `Material` converts a recipe into the bulk-composition vector **b**. Here the same basalt glass is expressed three equivalent ways, to show the flexibility of `Material`:

1. **element moles** — total iron lumped as ferrous (Fe), scaled to 100 g;
2. **oxide blend by mass** — grams of each oxide (with FeO and Fe_2O_3 listed separately);
3. **oxide blend by moles** — moles per formula unit (the version carried downstream as basalt).

Iron. This chemical system tracks *total* iron through a single Fe component, so both FeO and Fe_2O_3 feed that same component (no separate Fe|3| redox component is used here). The only bookkeeping difference between the all-ferrous element recipe (O = 3.320) and the FeO + Fe_2O_3 oxide recipe (O = 3.326) is the extra oxygen carried by the ferric fraction. set_quantity / scale_to_mass put any of these recipes on an absolute basis.

In [14]:
```
# ---- 4a. basalt as ELEMENT moles (total Fe, all as FeO) ----------------
basalt_elements = {
    "K":     0.008,
    "Na":    0.080,
    "Ca":    0.270,
    "Mg":    0.260,
    "Ti":    0.020,
    "Fe":    0.181,   # total iron -> FeO
    "Al":    0.350,
    "Si":    1.000,
    "O":     3.320,
}

basalt_el = Material(gems, "basalt_glass_elements")
basalt_el.add(basalt_elements)         # dict of {element: moles}, taken as absolute
basalt_el.set_quantity(100, "g")      # put the recipe on a 100 g basis
```

Out[14]: Material('basalt_glass_elements', 9 constituents, 100 g, 4.53618 mol)

In [15]:
```
basalt = Material(gems, "basalt_glass")
basalt.add("SiO2",   50,   "g")   # Silica
basalt.add("Al2O3",  15,   "g")   # Alumina
basalt.add("CaO",    12,   "g")   # Lime
basalt.add("FeO",    10,   "g")   # Ferrous oxide
basalt.add("MgO",     8.7, "g")   # Magnesia
basalt.add("Na2O",    2,   "g")   # Soda
basalt.add("TiO2",    1.3, "g")   # Titania
basalt.add("Fe2O3",   0.8, "g")   # Ferric oxide
basalt.add("K2O",     0.3, "g")   # Potash
```

Out[15]: Material('basalt_glass', 9 constituents, 100.1 g, 1.60505 mol)

In [16]:
```
# another variant

"""Basalt glass expressed as an oxide blend (moles per formula unit)."""
basalt = Material(gems, "basalt_glass")
basalt.add("SiO2",  1.000, "mol")
basalt.add("Al2O3", 0.175, "mol")
basalt.add("FeO",   0.169, "mol")
basalt.add("CaO",   0.270, "mol")
basalt.add("MgO",   0.260, "mol")
basalt.add("Na2O",  0.040, "mol")
basalt.add("K2O",   0.004, "mol")
basalt.add("TiO2",  0.020, "mol")
basalt.add("Fe2O3", 0.006, "mol")
basalt
```

Out[16]: Material('basalt_glass', 9 constituents, 121.101 g, 1.944 mol)

5. Build a Material — seawater

Seawater is built the geochemist's way — **1 kg of H_2O as the solvent plus the dissolved major ions**

supplied as an element dict (a Millero-style major-ion composition) — then scaled to 1 kg of solution with `set_quantity`.

Because the ions are supplied directly as elements, charge balance is not imposed by hand: GEMS reconciles it through its charge component (`Zz`) when the system is equilibrated.

In [17]:
```
"""Millero seawater — H2O as the solvent + dissolved ions as a dict."""
seawater = Material(gems, "seawater")
seawater.add("H2O", 1.00, "kg")
seawater.add(
        {
            "Cl": 1.55,  "Na": 1.47,
            "Mg": 0.053, "Ca": 0.01,
            "K":  0.010, "S":  0.028,
            "C":  0.002,
        }
    )
```

Out[17]:
```
Material('seawater', 8 constituents, 1091.75 g, 58.6314 mol)
```

In [18]:
```
seawater.set_quantity(1.0, "kg")   # scale to 1 kg of seawater
```

Out[18]:
```
Material('seawater', 8 constituents, 1000 g, 53.7041 mol)
```

In [19]:
```
with open("result_seawater.txt", "w") as f:
    f.write(repr(gems))          # same content as GEM-Selektor's multi-page print
```

6. Suppress the Brucite and Dolomite phases and re-check saturation

Setting a phase's **upper metastability limit to 0** is the GEM-Selektor `Upper_KC = 0` operation; in the API it is `suppress_phase(name, max_amount=0.0)` (or `suppress_multiple_phases([...])`). We equilibrate 1 kg of seawater + 5 g $CaCO_3$ at 250 °C / 1000 bar, suppress **Brucite** and **Dolomite**, re-equilibrate, and then read their saturation indices — which are now free to rise above 0.

> Adjust the phase names to match *your* database exactly.

In [21]:
```
mix = seawater(1.0, "kg")
mix.add("CaCO3",5,"g")
```

Out[21]:
```
Material('seawater(1 kg)', 9 constituents, 1005 g, 53.754 mol)
```

In [22]:
```
T = 250.0 + 273.15
P = 1000e5                       # 1000 bar
gems.equilibrate(T, P, mix)
# gems
```

Out[22]:
```
'OK after GEM calculation with LPP AIA'
```

In [23]:
```
phases_to_supress = ["Brucite", "Dolomite"]

gems.suppress_multiple_phases(phases_to_supress, min_amount=0.0, max_amount=0.0)
status = gems.reequilibrate()
print("status after suppression:", status)

# saturation indices of the suppressed phases (now free to go > 0)
si = gems.phase_sat_indices

# tabulate just the suppressed phases and their saturation indices
SI_table = pd.DataFrame({"SI (log10 IAP/K)": {p: si[p] for p in phases_to_supress if
SI_table.index.name = "Phase"
```

```
SI_table
```

```
status after suppression: OK after GEM calculation with LPP AIA
```

Out[23]:

Phase	SI (log10 IAP/K)
Brucite	3.696515
Dolomite	0.462533

7. Add CO_2 and read the gas fugacity

We inject CO_2 into the equilibrated seawater and read the fugacity of CO_2 in the gas phase. In GEMS the activity of a gas-phase species **is** its fugacity in bar, so `fugacity_bar = exp(species_ln_activities[gas_species])`. The partial pressure follows from the gas-phase mole fraction × total pressure.

In [30]:
```
gas_phase = gems.gas_phase_symbol

# add CO2 to the current (seawater) system and re-equilibrate
gems.add_amt_from_formula({"C": 1, "O": 2}, 0.05, "mol")   # 0.05 mol CO2
gems.reequilibrate()

# Return {gas_species: fugacity_bar} for the gas phase, using the GEMS convention
# that a gas species' activity equals its fugacity in bar.
ln_a = gems.species_ln_activities

fug = {sp: float(np.exp(ln_a[sp])) for sp in gems.species_in_phase[gas_phase]}

print("gas-phase fugacities (bar):")
for sp, f in fug.items():
    print(f"  {sp:12s} {f:12.4g}")
```

```
gas-phase fugacities (bar):
  CO              0.0003941
  CO2                 34.26
  CH4                  1.28
  HCl             5.945e-06
  H2                0.05314
  H2O                 50.55
  O2               2.92e-38
  SO2             9.665e-11
  H2S                 1.957
```

8. Hydrothermal extras

8a. Saturation index of key minerals vs temperature (fixed P)

Re-equilibrate the **seawater + a little basalt** mix along a T ramp and watch which secondary minerals cross into supersaturation (SI ≥ 0).

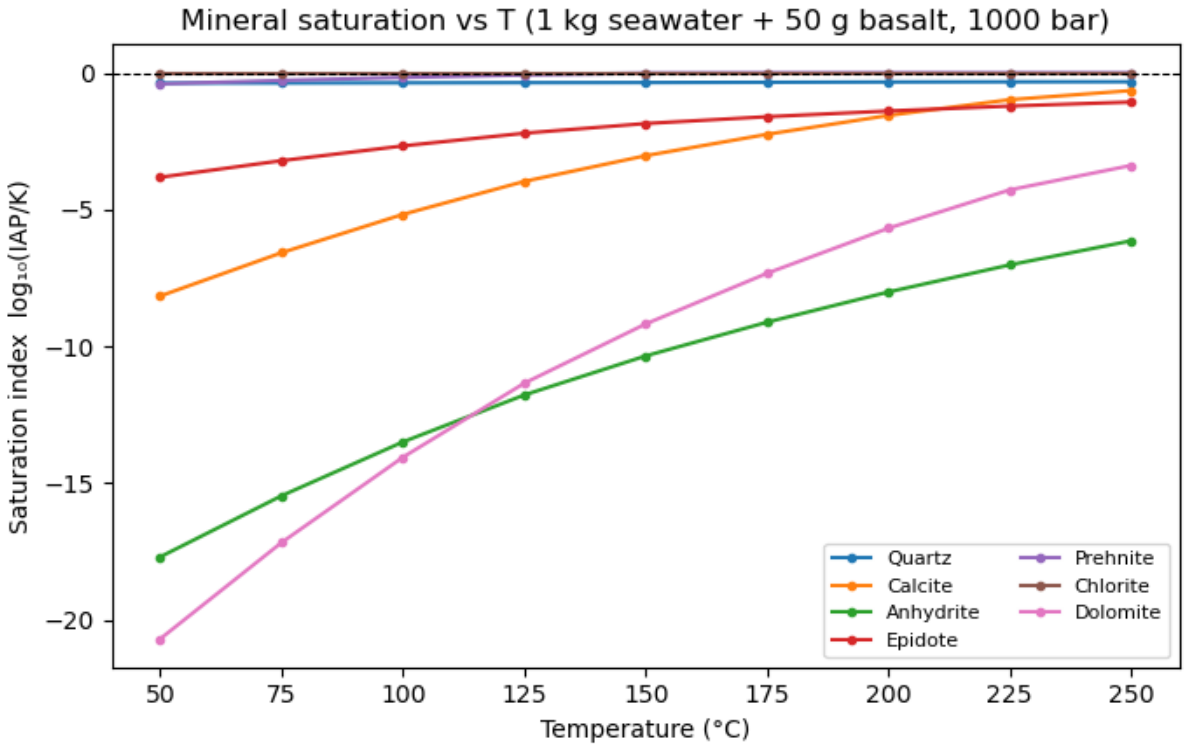
In [ ]:
```
# a modest reacted mix: 1 kg seawater + 50 g basalt glass
mix_b = seawater+basalt.scale_to_mass(0.050)

track = ["Quartz", "Calcite", "Anhydrite", "Epidote", "Prehnite", "Chlorite", "Dolomi
temps_C = np.arange(50, 251, 25)
P_fixed = 1000e5

rows = []
for tC in temps_C:
    gems.equilibrate(tC + 273.15, P_fixed, mix_b)
    si = gems.phase_sat_indices
```

```
    rows.append({m: si.get(m, np.nan) for m in track})
si_T = pd.DataFrame(rows, index=temps_C)
si_T.index.name = "T_degC"

# ploting
plt.figure(figsize=(7, 4.5))
for m in track:
    #if si_T[m].notna().any():
    plt.plot(si_T.index, si_T[m], marker="o", ms=3, label=m)
plt.axhline(0, color="k", lw=0.8, ls="--")
plt.xlabel("Temperature (°C)"); plt.ylabel("Saturation index  log₁₀(IAP/K)")
plt.title("Mineral saturation vs T (1 kg seawater + 50 g basalt, 1000 bar)")
plt.legend(fontsize=8, ncol=2); plt.tight_layout(); plt.show()
```

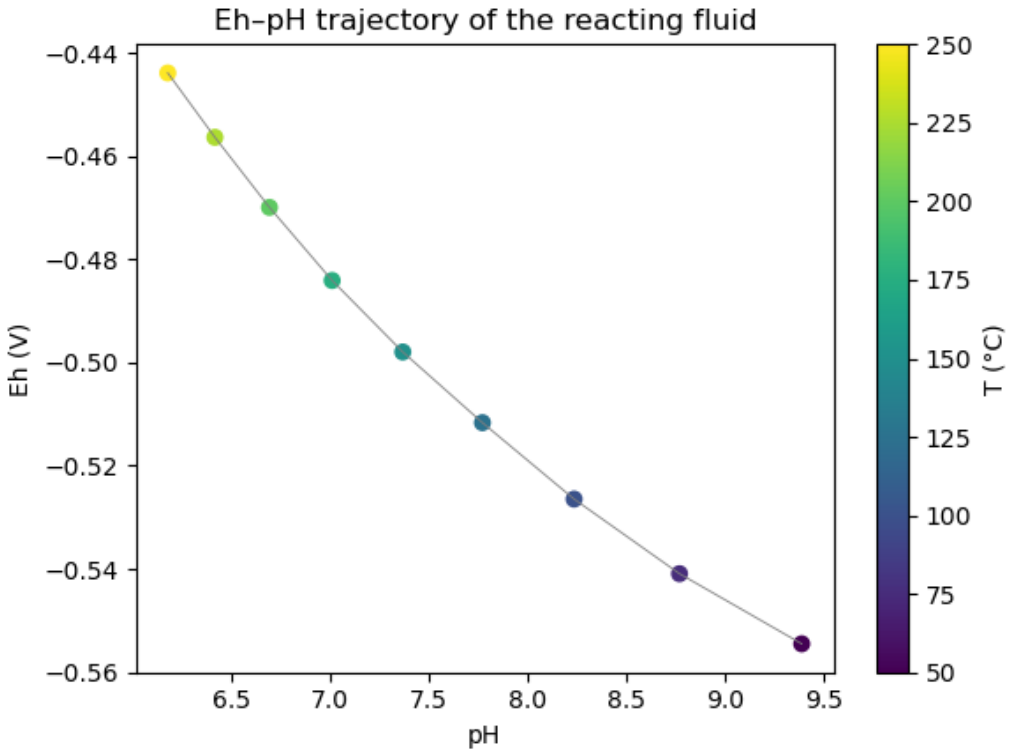


8b. Eh–pH of the reacting fluid along the same T ramp

A compact redox/acidity trajectory — where does basalt–seawater interaction drive the fluid on the Eh–pH plane as it heats up?

In [36]:
```
eh, ph = [], []
for tC in temps_C:
    gems.equilibrate(tC + 273.15, P_fixed, mix_b)
    eh.append(gems.Eh); ph.append(gems.pH)

plt.figure(figsize=(6, 4.5))
sc = plt.scatter(ph, eh, c=temps_C, cmap="viridis")
plt.plot(ph, eh, lw=0.6, color="grey")
plt.colorbar(sc, label="T (°C)")
plt.xlabel("pH"); plt.ylabel("Eh (V)")
plt.title("Eh–pH trajectory of the reacting fluid")
plt.tight_layout(); plt.show()
```



8c. Dominant aqueous speciation of a chosen element

Which complexes actually carry an element in solution? Here, the aqueous species of carbon at the current (final) state, ranked by molality.

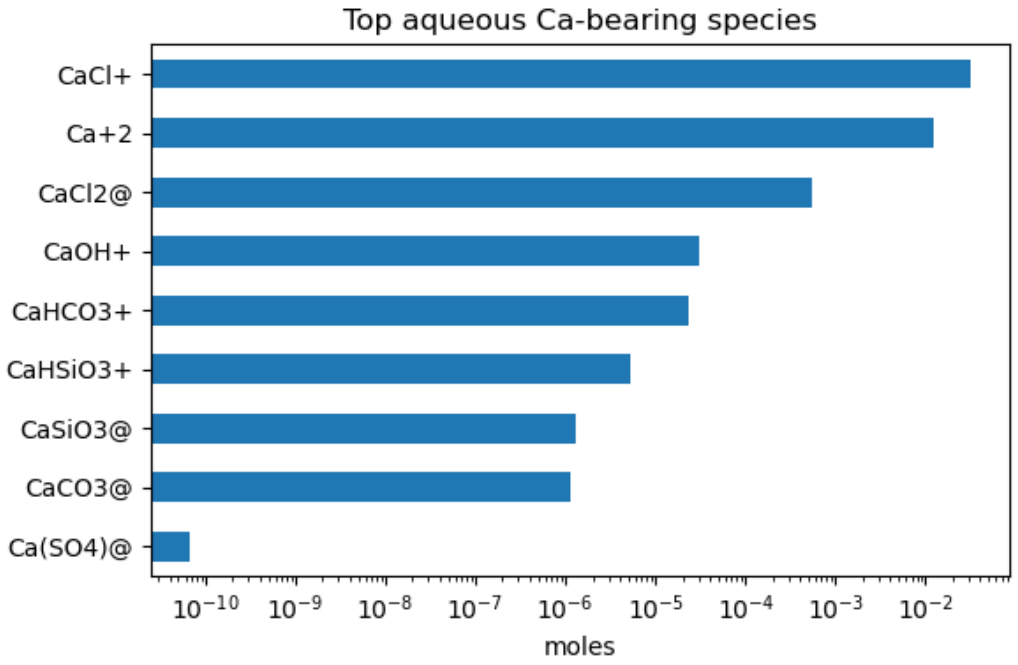
In [ ]:
```
gems.equilibrate(250 + 273.15, P_fixed, mix_b)

element_of_interest = "Ca"
sp_moles   = gems.species_moles
sp_charges = gems.species_charges
aq_phase = gems.aq_phase_symbol

aq_species = gems.species_in_phase[aq_phase]

# keep species whose formula/name contains the letter symbol
# more complex search is needed for elements like C that will have false positives (e
ca_sp = {sp: sp_moles[sp] for sp in aq_species
             if element_of_interest in sp and sp_moles[sp] > 0}
ser = pd.Series(ca_sp).sort_values(ascending=False).head(10)

plt.figure(figsize=(6, 4))
ser.iloc[::-1].plot(kind="barh")
plt.xscale("log"); plt.xlabel("moles"); plt.title(f"Top aqueous {element_of_interest}
plt.tight_layout(); plt.show()
ser
```



Out[ ]:
```
CaCl+       3.139956e-02
Ca+2        1.248792e-02
CaCl2@      5.378580e-04
CaOH+       3.094315e-05
CaHCO3+     2.342824e-05
CaHSiO3+    5.180705e-06
CaSiO3@     1.318355e-06
CaCO3@      1.159856e-06
Ca(SO4)@    6.632886e-11
dtype: float64
```

End of tutorial 1. Tutorial 2 uses these same `Material` builds to run a basalt→seawater titration at 250 °C / 1000 bar and plot phase volumes.